M5：MCP 工具接入

JSON-RPC 请求关联、生命周期和适配设计

M5：MCP 工具接入
- 背景、目标与范围
- 行为规格
  - M5-M-S01：配置与进程启动
  - M5-M-S02：消息分帧
  - M5-M-S03：请求关联
  - M5-M-S04：握手
  - M5-M-S05：工具列表
  - M5-M-S06：调用与结果
  - M5-M-S07：超时、取消和断线
  - M5-M-S08：工具名称映射
- 公共数据结构与端口
  - API 合同
- 详细设计与模式
- 验收轨迹
- 请求表和生命周期推演
  - 取消竞态
  - 分页的原子发布
  - 名称碰撞
  - 外部内容与协议错误
- 备选方案与协议参考

M5：MCP 工具接入

属性	内容
状态	设计提案 1.0
前置	M1工具合同；M4进程清理
协议基线	MCP 2025-11-25 的 stdio 工具子集
交付	`@local/mcp` 的 client、transport 和 adapter
验收	M5-M01 至 M5-M10

背景、目标与范围

本地工具不应为每个外部程序编写专门的 Core 逻辑。本模块使用 MCP 发现和调用工具，再适配为同一种 AgentTool。连接和协议状态由客户端拥有，Core 不知道 JSON-RPC 请求 ID。

基线支持 initialize、tools/list 分页、tools/call、ping 和取消通知，传输为 stdio 与测试用内存连接。非目标包括 HTTP、OAuth、sampling、elicitation、resources、prompts、任务型异步调用和自动重连。此实现是明确声明能力的协议子集，不宣称完整 MCP 支持。

行为规格

M5-M-S01：配置与进程启动

配置形状为 `{servers:Record<string,ServerConfig>}`。server name 需匹配 `[a-z0-9][a-z0-9_-]{0,31}`。用户配置从 agentHome/mcp.json 读取，可信项目配置从 workspace/.terminal-agent/mcp.json 读取。项目同名覆盖整个 server 配置，不合并 env 的局部字段；未信任时不读取项目文件。解析失败保留诊断并禁用该配置源，不隐式启动其他默认命令。

```
type ServerConfig = {
  command: string;
  args?: string[];
  env?: Record<string, string>;
  cwd?: string;
  timeoutMs?: number;
};
```

command 非空，args 缺省 []，timeout 默认 30000、正整数且不超过 120000；cwd 相对 workspace 解析，必须存在。不通过 shell 拼接，`spawn(command,args,{cwd,env})`。继承宿主环境再覆盖 env，但不记录值；启动此配置表示授权该进程拥有宿主用户权限。退出、超时和 close 均回收子进程。server stdout 只能是协议；stderr 进入有界诊断缓冲。

M5-M-S02：消息分帧

传输以 LF 分隔单行 UTF-8 JSON；JSON 字符串内的换行由转义表示，不是真实 LF。一次 chunk 可能包含多行，UTF-8 字符也可能跨 chunk，必须增量解码。允许 CRLF，去行尾 CR，不 trim JSON 字符串内容。

单行超过 1 MiB、无效 UTF-8、非法 JSON、数组 batch 或结构不合法：关闭连接并以 PROTOCOL_ERROR 结清 pending。EOF 尾部非空且无 LF 视为截断协议消息，不能猜测处理。空白行忽略并诊断；不误当响应。

写入使用单一队列，JSON.stringify 后加 LF，遵守 stdin.write=false 的 drain。不得同时拼接多次字符串而越过背压。日志和工具输出只能在 JSON 结果或 stderr，不能混进 stdout。

M5-M-S03：请求关联

客户端生成递增正整数请求 ID，连接内不复用。`pending:Map<number,Pending>` 记录 resolve/reject、timer、abort cleanup 和 method。必须先放进 Map 再 send，防止内存传输同步返回时丢响应。send 失败立即删除并拒绝。

响应恰好含 result 或 error 其中一个；id 必须等于一个 pending ID。匹配后先删除、清 timer/abort listener，再结算 Promise。未知/迟到/重复响应不再次结算，给 diagnostic。对方请求 ID 可以是 string 或 number；回复原 ID。通知没有 id，不能创建 pending 或等一个响应。

服务器 ping 请求可回空 result。不支持的服务器请求回 JSON-RPC -32601。不支持通知仅诊断，不回 response。不得把通知错误当调用失败广播给所有 pending。

M5-M-S04：握手

连接状态：idle → connecting → connected → closed。connect 只能从 idle 开始，第二次 connect 是 INVALID_INPUT；失败关闭，重连须新建 client。

1. 连接 transport，设置接收回调后发送 initialize。
2. params 包含 protocolVersion=`2025-11-25`、capabilities=`{}`、clientInfo=`{name:'terminal-agent',version:'1.0.0'}`。
3. 校验返回的 protocolVersion。基线只接受 `2025-11-25`；不支持版本关闭并 PROTOCOL_ERROR。不假装已兼容其他版本。
4. 保存 serverInfo/capabilities，发送 notifications/initialized，写入成功后进入 connected。
5. capabilities.tools 存在才允许自动 tools/list；否则该服务提供零工具。

initialize 超时不得继续 list。connecting 时除内部 initialize 和 ping 外的请求拒绝 INVALID_INPUT。能力声明为 {} 是因为本客户端不实现 roots/sampling/elicitation；不能宣传没有的能力。

M5-M-S05：工具列表

`listTools()` 在 connected 且有 tools 能力时发送 tools/list。服务器返回 tools 数组与可选 nextCursor；有 cursor 则带 cursor 请求下一页，直到不存在。最多 100 页、1000 工具；cursor 重复视为 PROTOCOL_ERROR，防止无限循环。

每个工具须有非空 name、合法对象 inputSchema，description 可空。同 server 重复原名使该次列表整体失败，不发布部分注册表。客户端支持的 schema 方言由 M1 校验器决定；不支持的单个工具跳过并诊断。列表刷新在 Session idle、统一 reload 中进行；运行中的 listChanged 通知只设置 dirty 标志，下一 idle 才重新获取。

M5-M-S06：调用与结果

`callTool(originalName,args,{signal,timeoutMs})` 发送 tools/call，params=`{name:originalName,arguments:args}`。必须使用名称映射中的原始 server 工具名，不能把模型侧规范化名发给服务器。

基线可显示 text content，image/resource 等块转为 `[unsupported content: <type>]` 提示并保留 types 列表；不能把二进制数据误当文本。structuredContent 若为合法 JSON，保存在 details。服务器 result.isError=true 是工具业务错误，映射为 TOOL_FAILED；JSON-RPC error 是请求失败，也由 adapter 转成错误 ToolOutcome。两者都不杀死其他工具。

输出仍遵守 M4 截断预算。结果无 content 时允许空字符串；字段结构错返回 PROTOCOL_ERROR 的该次工具结果，并关闭连接以避免继续消费损坏协议。

M5-M-S07：超时、取消和断线

request timeout 默认 30000ms，可按单次工具配置覆盖。超时或 signal 取消：先从 pending 删除并清理，再发送 `notifications/cancelled`，params=`{requestId,reason}`（仅对已发送且非 initialize 的请求）；本地立即以 TIMEOUT/ABORTED 拒绝。取消通知是尽力而为，失败写到诊断，不重新结算。

迟到响应只诊断。断线把全部 pending 以 DISCONNECTED 拒绝一次，清 timer/listener，进入 closed。close 幂等，拒绝新请求并关闭 transport；stdio 关闭 stdin、等待退出，1000ms 后 SIGTERM，再1000ms后 SIGKILL并等 close。

不自动重试 tools/call，不自动重连后重放 pending。服务端可能已经完成副作用，本地取消不能证明远端回滚。M2 恢复规则仍适用。

M5-M-S08：工具名称映射

模型侧名称必须满足 M1 的 64 字符限制。生成基础名 `mcp_<server>_<tool>`，把非 `[A-Za-z0-9_-]` 字符替换为 `_`。若超长或与已有名字冲突，保留前 47 字符，加 `_` 和 SHA-256(server + NUL + originalName) 前 16 hex，共最多64字符。

对含冲突的候选按 server、原名码点顺序注册，保证确定性。若 hash 后仍冲突，继续使用 SHA-256 前 24 hex 并相应缩短前缀；仍冲突则注册失败并诊断，绝不覆盖已有工具。`nameToRemote:Map<string,{server,originalName}>` 是调用权威；不要从下划线拆回原名。

所有 MCP AgentTool 的 effect=external，不能相信服务器声称只读就自动并行。若未来增加受信工具标注，需要单独审查规则。

公共数据结构与端口

```
type RpcId = string | number;
type RpcRequest = {
  jsonrpc: "2.0"; id: RpcId; method: string; params?: Json;
};
type RpcNotification = {
  jsonrpc: "2.0"; method: string; params?: Json;
};
type RpcResponse =
  | { jsonrpc: "2.0"; id: RpcId; result: Json }
  | { jsonrpc: "2.0"; id: RpcId;
      error: { code: number; message: string; data?: Json } };
interface Transport {
  start(handlers: {
```

```
    message(value: unknown): void;
    close(error?: Error): void;
  }): Promise<void>;
  send(value: RpcRequest | RpcNotification | RpcResponse): Promise<void>;
  close(): Promise<void>;
}
type RemoteTool = {
  name: string;
  description?: string;
  inputSchema: Json;
};
type Pending = {
  method: string;
  resolve(value: Json): void;
  reject(error: Error): void;
  dispose(): void;
};
```

协议包内可独立定义 Json 或导入零领域依赖的类型包；不能让 jsonrpc/client 为了 Json 导入 Core。仅 adapter 将 RemoteTool 转为 AgentTool 并归一错误。

API 合同

API	前置	返回
connect()	idle	握手和 initialized 写入成功
listTools()	connected	全部有效工具与诊断
callTool(name,args,options)	connected	原始工具结果或领域错误
request(method,params,options)	符合生命周期	对应 result
close()	任意	transport 与 pending 已清理
createInMemoryPair()	无	双端 transport；send 深拷贝，支持断线注入

详细设计与模式

端口与适配器：client 只依赖 Transport；stdio 处理进程和分帧，in-memory 处理测试。不让 client 依赖 child_process，才能用相同握手测试真实与假传输。

请求关联表：Map 让乱序响应平均 O(1) 找到等待者。每个 pending 有统一 settle/dispose，所有响应、取消、超时和断线都调用同一删除路径，保证不泄漏 timer。每条连接的额外内存 O(p + 工具列表 + 有界缓冲)。

适配器：MCP 响应与 Core ToolOutcome 分开，adapter 负责名称映射、内容转换、错误 code 和 effect。Core 不根据工具名中的 mcp 前缀改变执行规则。

快照发布：reload 建新工具列表并完整验证，再在 Session idle 更新工具快照与 prompt。失败保留旧列表；连接已经断开的旧工具可保留为明确 DISCONNECTED 的禁用项，下一次成功刷新再替换。不得重用旧名称映射去调用不同原名。

验收轨迹

```
client -> initialize(id=1)
server -> result(id=1, protocolVersion=2025-11-25, capabilities.tools={})
client -> notifications/initialized
client -> tools/list(id=2)
server -> result(id=2, tools=[...])
client -> tools/call(id=3, originalName, arguments)
server -> result(id=3, content=[text])
```

编号	场景	断言
M5-M01	内存对端握手、工具列出并调用	顺序与轨迹一致；原名正确
M5-M02	服务返回不支持版本/initialize 超时	关闭；无 list/call
M5-M03	2 个请求，响应反序	分别 resolve 正确等待者；pending 清空
M5-M04	请求发送前同步响应、send 失败	无丢响应；失败清 pending
M5-M05	chunk 拆中文/多行、超过行限、EOF 半行	正确解码；损坏时统一关闭
M5-M06	timeout/abort 后迟到响应	只结算一次；发取消；无 timer 泄漏
M5-M07	连接关闭时存在多个 pending	全部 DISCONNECTED；close 幂等
M5-M08	多页/重复 cursor/重复工具	完整列表发布；非法不半发布
M5-M09	长名、非法字符、规范化冲突	名称<=64且唯一；映射原名不丢
M5-M10	stderr 日志、text/非文本/isError、服务器未知请求	日志不进协议；降级内容明确；回 -32601

至少一条 stdio 集成测试启动项目内的假服务器，覆盖真实分帧和退出清理；其余用内存传输和 fake clock。

请求表和生命周期推演

取消竞态

request id=7已经写出，signal和response在同一个事件循环回合先后到来。第一个成功从pending删除者拥有结算权。若response先到则resolve并移除abort listener；若abort先到则reject ABORTED、尽力发cancelled，之后response只记迟到诊断。不能让同一个Promise先成功后又发一个业务失败事件。

initialize不发cancelled通知，超时直接关闭连接。notifications/initialized没有id，也不能出现在pending里。与server的ping请求ID同值并不冲突：入站请求和出站pending是两个方向，不能共享一个不区分方向的Map。

分页的原子发布

第1页返回tool A，cursor=X；第2页超时。对外不发布“只有A”的不完整新表，保持旧工具快照并诊断刷新失败。初次连接没有旧表则该服务零工具，但其他服务可以独立成功。一个服务断线不关闭整个Agent；只有调用到它的工具返回DISCONNECTED。

名称碰撞

server=s、tool=`read/file`与tool=`read_file`都规范化为`mcp_s_read_file`。按原名排序后第一个可用基础名，第二个追加hash；映射分别保存原字符串。远端调用不根据规范化名猜回斜线。工具重载后对同一组输入必须得到同一映射，不能依赖并发服务器返回先后。

外部内容与协议错误

服务器返回isError=true是有效结果，连接可以继续用；返回无法解析的JSON是传输协议损坏，连接关闭。服务器工具文字中包含一段看似JSON-RPC的字符串仍只是content，不能再次送进协议解析器。stdio日志误写stdout会导致PROTOCOL_ERROR，这是明确可诊断的服务错误，不靠正则“挑出看似正确行”隐藏。

传输实例拥有自己的buffer与handler，关闭后丢弃迟到chunk，不把它们误送给新连接。内存传输也必须复制消息，防止测试中的对象共享掩盖真实序列化边界。

备选方案与协议参考

直接给 Core 增加“远程工具”分支会耦合协议生命周期，因此选普通 AgentTool 适配。固定接受一个协议版本比未经验证地宣称支持多个版本更适合独立实现。HTTP 和重连可以后续增加，但必须保留请求关联与不重放副作用的合同。

外部协议以官方 生命周期、stdio 传输 与 工具为准；本页的超时、命名、分页预算、只接受单版本等属于本项目的实现选择。先实现内存传输与关联表，再握手、工具适配，最后 stdio 和进程清理。